

Linguagem Funcional 1

- Linguagem Funcional de Primeira Ordem
- Declarações
- Implementações

Linguagem Funcional de Primeira Ordem

- A linguagem lf_1 estende a Linguagem de Expressão 2 (le_2) com funções parametrizadas e recursivas (de primeira ordem, funções não são valores).
- A sintaxe da Linguagem Funcional 1 (lf_1) é apresentada na figura a seguir.

Linguagem Funcional de Primeira Ordem

```
Programa ::= Expressao
Expressao ::= Valor | ExpUnaria | ExpBinaria | ExpDeclaracao | Id |
           Aplicacao | IfThenElse
ExpDeclaracao ::= "let" DeclaracaoFuncional "in" Expressao
DeclaracaoFuncional ::= DecVariavel | DecFuncao |
                        DeclaracaoFuncional "," DeclaracaoFuncional
DecFuncao ::= "fun" ListId "=" Expressao
Aplicacao ::= Id "(" ListExp ")"
IfThenElse ::= "if" Expressao "then" Expressao "else" Expressao
DecVariavel ::= ...
Valor ::= ...
ExpUnaria ::= ...
ExpBinaria ::= ...
```

Figura 17. BNF para a Linguagem de Funcional 1

Linguagem Funcional de Primeira Ordem

- O corpo de uma função é uma expressão e a aplicação da função a um argumento retorna um valor
- A aplicação da função também é uma expressão.
- um programa nesta linguagem também é uma expressão
- que pode envolver invocação (aplicação) de função.

Linguagem Funcional de Primeira Ordem

- Além da introdução de definição e aplicação de função, a linguagem inclui uma expressão condicional (if ... then ... else).
- Apesar de ser mais usual como um comando, o comportamento aqui é semelhante.
- O uso de expressões condicionais é essencial na definição de funções recursivas que tipicamente incluem um caso base e o caso geral.

Declarações

- Declaração de função ocorre em blocos de declaração.
- Na Linguagem de Expressão 2 (le2), um bloco pode conter apenas declaração de variáveis;
- Em lf1, a construção é estendida para permitir também declaração de funções.

Declarações

- Um mesmo bloco pode conter os dois tipos de declaração: função e aplicação.
- A declaração de função inclui uma lista de identificadores (os parâmetros da função)
- e um corpo, constituído da expressão a ser avaliada quando a função é invocada.

Declarações

- A aplicação de função envolve o identificador da função invocada
- e uma lista de expressões, que deve obedecer a ordem e os tipos dos parâmetros.

Declarações

- Um exemplo de programa nesta linguagem é
- `let fun f x = x + 1 in f 2`
- Observem que a expressão declaração já é um programa completo,
- inclusive com os dados para serem calculados (aplicação da função).

Declarações

- Os blocos podem ser aninhados, como na le2, só que agora podem incluir declarações de variáveis e funções, por exemplo
- `let var x = 3 in`
- `let fun f y = y + x in`
- `let var x = 5 in`
- `f 1`

Declarações

- Um outro exemplo de programa nesta linguagem é o bloco
- `let fun mult x y = if x == 0 then 0 else y + mult (x - 1, y) in mult (3,4)`
- que introduz uma definição recursiva da multiplicação de dois inteiros usando soma.
- Na definição, o produto de x e y é obtido pela soma do valor de y um número de vezes dado pelo valor de x .

Declarações

- Um outro exemplo de programa nesta linguagem é
- `let fun mult x y = if x == 0 then 0 else y + mult(x-1, y) in`
 - `let fun square x = mult(x, x) in square (7)`
- computa o quadrado de `y` (com valor 7), usando a função `mult`. O resultado da avaliação do programa inteiro é 49.

Declarações

- O tipo de uma função é inferido automaticamente a partir das declarações e expressões do programa.
- O tipo inferido é o mais geral possível para os parâmetros e resultado da função.

Declarações

- Um exemplo simples é a função identidade, que é polimórfica já que não há qualquer restrição no argumento e resultado da função.
- `let fun id x = x in id(2)`
- No caso, a função é aplicada a um argumento inteiro, mas argumentos de quaisquer tipos podem ser utilizados.

Declarações

- Como esta linguagem oferece duas formas de declaração (variáveis e funções),
- os ambientes de execução e de compilação precisam incluir mapeamentos que representem e armazenem as respectivas informações.

Declarações

- O ambiente de execução inclui dois componentes:
- um mapeamento de identificadores em valores (como na Linguagem de Expressão 2)
- e um mapeamento de identificadores (nomes de função) em definições de função.

Declarações

- O ambiente de compilação possui um mapeamento de identificadores em tipo,
- mas o conceito de tipo é estendido para funções, que é representado pelo produto cartesiano dos tipos dos parâmetros e o tipo do resultado produzido pela função.

Declarações

- A interface que modela o ambiente de execução é apresentada na Figura a seguir.
- Analogamente aos métodos para mapear uma declaração e retornar o valor de um identificador,
- a interface estendida adiciona métodos para mapear uma declaração de função e retornar a definição de uma função associada a um dado identificador.

Declarações

```
public interface AmbienteExecucaoFuncional extends AmbienteExecucao {  
    public void mapFuncao(Id idArg, ValorFuncao funcao) throws VariavelJaDeclaradaException;  
    public ValorFuncao getFuncao(Id idArg) throws VariavelNaoDeclaradaException;  
}
```

Figura 18. Interface do ambiente de execução funcional

Implementações

- A classe mais elaborada da Linguagem Funcional 1 é a que implementa aplicação de função que destaca os atributos da classe e o método responsável pela avaliação.
- Conforme código na Figura a seguir.

Implementações

```
public class Aplicacao implements Expressao {
    private Id func;
    private List<Expressao> argsExpressao;
    public Valor avaliar(AmbienteExecucao ambiente) {
        AmbienteExecucaoFuncional ambienteFuncional =(AmbienteExecucaoFuncional) ambiente;
        ValorFuncao funcao = ambienteFuncional.getFuncao(func);
        Map<Id,Valor> mapIdValor = resolveParametersBindings(ambiente, funcao);
        ambiente.incrementa();
        includeValueBindings(ambiente, mapIdValor);
        Valor vresult = funcao.getExp().avaliar(ambiente);
        ambiente.restaura();
        return vresult;
    }
}
```

Figura 19. Código da classe Aplicação

Implementações

- O primeiro atributo representa o identificador da função invocada.
- O segundo é uma lista de expressões que representa os argumentos da invocação.
- A implementação da avaliação da aplicação é semelhante à avaliação de um bloco de declarações let

Implementações

- A primeira linha do corpo do método avaliar efetua um cast no ambiente de execução (parâmetro do método) requerendo que o ambiente seja de fato um ambiente de execução funcional.
- A idéia de manter o parâmetro de avaliar com tipo AmbienteExecucao e realizar o cast é que isto permite a reutilização das interfaces da linguagem le2.

Implementações

- A segunda linha do método recupera a definição da função, no ambiente, associada ao identificador da função invocada.
- A linha seguinte cria um mapeamento que representa a avaliação das expressões passadas como argumento para a função.

Implementações

- Em seguida, o ambiente é incrementado com este mapeamento,
- a expressão que representa o corpo da função é avaliada neste contexto,
- o ambiente é restaurado
- e o valor resultante da avaliação é retornado.

Atividade 1

- Usando a lf1, elabore uma classe em Java chamada de **Exemplo1** para avaliar a seguinte expressão:
- `let fun f x = x + 1 in f 2`

Atividade 2

- Usando a lf1, elabore uma classe em Java chamada de **Exemplo2** para avaliar a seguinte expressão:
- let var x = 3 in
- let fun f y = y + x in
- let var x = 5 in
- f 1

Atividade 3

- Usando a lf1, elabore uma classe em Java chamada de **Exemplo3** para avaliar a seguinte expressão:
- `let var y = 3 in`
- `let fun f x = x + y in`
- `let var z = "abc" in`
- `f 3`

Atividade 4

- Usando a lf1, elabore uma classe em Java chamada de **Exemplo4** para avaliar a seguinte expressão:
- `let fun mult x y = if (x == 0) then (0) else (y + mult (x - 1, y)) in mult (3,4)`

Subprojeto If1

- Converter toda a linguagem If1 que está escrita em Java para Python (inclusive com as classes de exemplos).
- Esse subprojeto completo deve ser entregue no final do curso.

Leitura recomendável

- Conceitos e Paradigmas de Programação via Projeto de Interpretadores
- Augusto Sampaio e Antônio Maranhão
- Material em preparação para o JAI-2008 (Jornadas de atualização em Informática – Sociedade Brasileira de Computação)